AMCAT (SHL) Automata Fix - 50 Corrected Questions

For CS Students | Difficulty: Moderate-High | Language: C/C++

Section 1: Compilation Errors (Syntax Fix)

Q1: Missing semicolon

Problem: Fix the syntax error to read and print an integer.

Buggy Code:
```
#include <stdio.h>
int main() {
int n
scanf("%d", &n);
printf("%d\n", n);
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main() {
int n; // Added semicolon
scanf("%d", &n);
printf("%d\n", n);
return 0;
}
```

Q2: Wrong operator in scanf

Problem: Fix to input a and b and print their sum.

Buggy Code:
```
#include <stdio.h>
int main() {
int a, b;
scanf("%d%d", &a >> &b); // Wrong: >> instead of comma
printf("%d", a + b);
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main() {
int a, b;
```

```
scanf("%d%d", &a, &b); // Corrected: use comma
printf("%d", a + b);
return 0;
}
```

Q3: Missing header, wrong brackets

Problem: Complete scanf and printf calls.

Buggy Code:
```
int main(){
int x;
scanf("%d" &x); // Missing comma, missing >
printf "%d\n", x); // Missing (
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
int x;
scanf("%d", &x); // Fixed comma
printf("%d\n", x); // Fixed brackets
return 0;
}
```

Q4: Data type mismatch in printf

Problem: Fix the format specifier mismatch.

Buggy Code:
```
#include <stdio.h>
int main(){
float x = 3.14;
printf("%d\n", x); // Wrong: %d for float
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
float x = 3.14;
printf("%f\n", x); // Corrected: %f for float
return 0;
}
```

Q5: Missing return statement + bracket mismatch

Problem: Complete the function structure.

Buggy Code:
```
int main() {
int a = 5;
printf("%d", a);
// Missing return statement
}
```

Corrected Code:
```
#include <stdio.h>
int main() {
int a = 5;
printf("%d", a);
return 0;
}
```

Q6: Missing include header

Problem: Input two numbers and add them.

Buggy Code:
```
int main() {
int a, b;
cin >> a >> b; // Missing #include <iostream>
cout << a + b;
return 0;
}
```

Corrected Code:
```
#include <iostream>
using namespace std;
int main() {
int a, b;
cin >> a >> b;
cout << a + b;
return 0;
}
```

Q7: Array declaration syntax error

Problem: Declare and print array elements.

Buggy Code:
```
#include <stdio.h>
int main(){
int arr[5] = 1,2,3,4,5; // Wrong syntax
printf("%d", arr[0]);
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
int arr[5] = {1,2,3,4,5}; // Corrected: use braces
printf("%d", arr[0]);
return 0;
}
```

Q8: String declaration and output

Problem: Fix syntax to print a string.

Buggy Code:
```
int main(){
char str[] = "Hello World"
printf("%s\n", str); // Missing semicolon above
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
char str[] = "Hello World"; // Added semicolon
printf("%s\n", str);
return 0;
}
```

Q9: Assignment vs comparison in condition

Problem: Fix the if condition syntax.

Buggy Code:
```
#include <stdio.h>
int main(){
int n = 5;
if(n = 5) printf("Equal"); // Assignment = instead of ==
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
int n = 5;
if(n == 5) printf("Equal"); // Corrected: use ==
return 0;
}
```

Q10: Missing closing bracket in loop

Problem: Fix loop syntax to print 1 to 5.

Buggy Code:
```
#include <stdio.h>
int main(){
for(int i=1; i<=5; i++
printf("%d ", i); // Missing closing bracket
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
for(int i=1; i<=5; i++) // Added closing bracket
printf("%d ", i);
return 0;
}
```

Section 2: Logical Errors (Loop & Condition Issues)

Q11: Loop condition reversed

Problem: Print numbers 1 to n.

Buggy Code:
```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
for(int i=1; i>=n; i++){ // Wrong: >= instead of <=
printf("%d ", i);
}
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
for(int i=1; i<=n; i++){ // Fixed: <=
printf("%d ", i);
}
return 0;
}
```

Q12: Off-by-one in array loop

Problem: Read n integers and print sum.

Buggy Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int arr[n];
for(int i=0; i<=n; i++){ // Wrong: i<=n exceeds bounds
scanf("%d", &arr[i]);
}
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int arr[n];
for(int i=0; i<n; i++){ // Fixed: i<n
scanf("%d", &arr[i]);
}
return 0;
}
```

Q13: Wrong increment in loop

Problem: Count digits of a number.

Buggy Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int cnt = 0;
while(n > 0){
cnt++;
n = n / 10; // Correct
}
if(cnt == 0) cnt = 1; // Handle 0 case
printf("%d", cnt);
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
if(n == 0){ printf("1"); return 0; }
int cnt = 0;
while(n > 0){
cnt++;
```

```
n = n / 10;
}
printf("%d", cnt);
return 0;
}
```

Q14: Even/Odd check with wrong operator

Problem: Check if number is even or odd.

Buggy Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
if(n % 2 = 0) printf("EVEN"); // Assignment = instead of ==
else printf("ODD");
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
if(n % 2 == 0) printf("EVEN"); // Fixed: ==
else printf("ODD");
return 0;
}
```

Q15: Wrong comparison logic for max

Problem: Find maximum of three numbers.

Buggy Code:

```
#include <stdio.h>
int main(){
int a, b, c; scanf("%d%d%d", &a, &b, &c);
int ans = a;
if(b > ans) ans = b;
if(c < ans) ans = c; // Wrong: < instead of >
printf("%d", ans);
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int main(){
int a, b, c; scanf("%d%d%d", &a, &b, &c);
int ans = a;
if(b > ans) ans = b;
if(c > ans) ans = c; // Fixed: >
printf("%d", ans);
```

```
return 0;
}
```

Q16: Factorial loop boundary

Problem: Calculate n! (0 to 12).

Buggy Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int fact = 1;
for(int i=1; i<n; i++){ // Wrong: i<n (skips n)
fact *= i;
}
printf("%d", fact);
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int fact = 1;
for(int i=1; i<=n; i++){ // Fixed: i<=n
fact *= i;
}
printf("%d", fact);
return 0;
}
```

Q17: Fibonacci loop condition

Problem: Print nth Fibonacci number (0-indexed).

Buggy Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int a=0, b=1, c;
for(int i=2; i<n; i++){ // Wrong: i<n (off-by-one)
c = a + b;
a = b; b = c;
}
printf("%d", c);
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int main(){
```

```
int n; scanf("%d", &n);
if(n == 0){ printf("0"); return 0; }
if(n == 1){ printf("1"); return 0; }
int a=0, b=1, c;
for(int i=2; i<=n; i++){ // Fixed: i<=n
c = a + b;
a = b; b = c;
}
printf("%d", c);
return 0;
}
```

Q18: Reverse number - wrong remainder operation

Problem: Reverse a number.

Buggy Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int rev = 0;
while(n > 0){
rev = rev * 10 + n % 10;
n = n % 10; // Wrong: should divide, not modulo
}
printf("%d", rev);
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int rev = 0;
while(n > 0){
rev = rev * 10 + n % 10;
n = n / 10; // Fixed: divide by 10
}
printf("%d", rev);
return 0;
}
```

Q19: Palindrome check with assignment

Problem: Check if number is palindrome.

Buggy Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int x = n, rev = 0;
```

```
while(x > 0){
rev = rev * 10 + x % 10;
x /= 10;
}
if(rev = n) printf("YES"); // Assignment = instead of ==
else printf("NO");
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int x = n, rev = 0;
while(x > 0){
rev = rev * 10 + x % 10;
x /= 10;
}
if(rev == n) printf("YES"); // Fixed: ==
else printf("NO");
return 0;
}
```

Q20: String length with overflow

Problem: Input word, print its length.

Buggy Code:

```
#include <stdio.h>
#include <string.h>
int main(){
char s[100];
scanf("%s", s);
printf("%d", strlen(s) + 1); // Wrong: +1 includes null terminator
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
#include <string.h>
int main(){
char s[100];
scanf("%s", s);
printf("%d", strlen(s)); // Fixed: just strlen
return 0;
}
```

Section 3: Array & String Problems

Q21: Count vowels with array overflow

Problem: Count vowels in a string.

Buggy Code:
```
#include <bits/stdc++.h>
using namespace std;
int main(){
string s; cin >> s;
int cnt = 0;
for(int i=0; i<=s.size(); i++){ // Wrong: <= exceeds bounds
char c = s[i];
if(c=='a'||c=='e'||c=='i'||c=='o'||c=='u') cnt++;
}
cout << cnt;
return 0;
}
```

Corrected Code:
```
#include <bits/stdc++.h>
using namespace std;
int main(){
string s; cin >> s;
int cnt = 0;
for(int i=0; i<s.size(); i++){ // Fixed: <
char c = s[i];
if(c=='a'||c=='e'||c=='i'||c=='o'||c=='u') cnt++;
}
cout << cnt;
return 0;
}
```

Q22: Sum of array with overflow loop

Problem: Read n numbers and find sum.

Buggy Code:
```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int arr[n];
long long sum = 0;
for(int i=0; i<=n; i++){ // Wrong: i<=n
scanf("%d", &arr[i]);
sum += arr[i];
}
printf("%lld", sum);
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int arr[n];
long long sum = 0;
for(int i=0; i<n; i++){ // Fixed: i<n
scanf("%d", &arr[i]);
sum += arr[i];
}
printf("%lld", sum);
return 0;
}
```

Q23: Max element with uninitialized variable

Problem: Find maximum in array.

Buggy Code:
```
#include <bits/stdc++.h>
using namespace std;
int main(){
int n; cin >> n;
vector<int> a(n);
for(int i=0; i<n; i++) cin >> a[i];
int mx = 0; // Wrong: if all numbers are negative, result is wrong
for(int i=0; i<n; i++){
if(a[i] > mx) mx = a[i];
}
cout << mx;
return 0;
}
```

Corrected Code:
```
#include <bits/stdc++.h>
using namespace std;
int main(){
int n; cin >> n;
vector<int> a(n);
for(int i=0; i<n; i++) cin >> a[i];
int mx = a[0]; // Fixed: initialize with first element
for(int i=0; i<n; i++){
if(a[i] > mx) mx = a[i];
}
cout << mx;
return 0;
}
```

Q24: Linear search with assignment

Problem: Find index of element x (return -1 if not found).

Buggy Code:
```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int a[1000];
for(int i=0; i<n; i++) scanf("%d", &a[i]);
int x; scanf("%d", &x);
int idx = -1;
for(int i=0; i<n; i++){
if(a[i] = x){ idx = i; break; } // Assignment = instead of ==
}
printf("%d", idx);
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int a[1000];
for(int i=0; i<n; i++) scanf("%d", &a[i]);
int x; scanf("%d", &x);
int idx = -1;
for(int i=0; i<n; i++){
if(a[i] == x){ idx = i; break; } // Fixed: ==
}
printf("%d", idx);
return 0;
}
```

Q25: Array input with wrong initialization

Problem: Print elements in reverse order.

Buggy Code:
```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
int a[n];
for(int i=0; i<n; i++) scanf("%d", &a[i]);
for(int i=n-1; i>=0; i--) // Correct
printf("%d ", a[i]);
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
```

```
int n; scanf("%d", &n);
int a[n];
for(int i=0; i<n; i++) scanf("%d", &a[i]);
for(int i=n-1; i>=0; i--)
printf("%d ", a[i]);
return 0;
}
```

Section 4: Math Problems

Q26: GCD with wrong variable assignment

Problem: Find GCD of two numbers (Euclidean algorithm).

Buggy Code:

```
#include <stdio.h>
int gcd(int a, int b){
while(b != 0){
int t = a % b;
a = b;
b = a; // Wrong: should be b = t
}
return a;
}
int main(){
int a, b; scanf("%d%d", &a, &b);
printf("%d", gcd(a, b));
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int gcd(int a, int b){
while(b != 0){
int t = a % b;
a = b;
b = t; // Fixed: b = t
}
return a;
}
int main(){
int a, b; scanf("%d%d", &a, &b);
printf("%d", gcd(a, b));
return 0;
}
```

Q27: Prime check with wrong loop boundary

Problem: Check if n is prime.

Buggy Code:
```
#include <bits/stdc++.h>
using namespace std;
int main(){
int n; cin >> n;
bool prime = true;
for(int i=2; i*i <= n; i++){ // Correct
if(n % i == 0){ prime = false; break; }
}
if(n <= 1) prime = false; // Handle edge case
cout << (prime? "PRIME" : "NOT PRIME");
return 0;
}
```

Corrected Code:
```
#include <bits/stdc++.h>
using namespace std;
int main(){
int n; cin >> n;
if(n <= 1){ cout << "NOT PRIME"; return 0; }
bool prime = true;
for(int i=2; i*i <= n; i++){
if(n % i == 0){ prime = false; break; }
}
cout << (prime? "PRIME" : "NOT PRIME");
return 0;
}
```

Q28: Prime check with < instead of <=

Problem: Check if number is prime.

Buggy Code:
```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
if(n < 2){ printf("NOT PRIME"); return 0; }
for(int i=2; i*i < n; i++){ // Wrong: < instead of <=
if(n % i == 0){ printf("NOT PRIME"); return 0; }
}
printf("PRIME");
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
```

```
if(n < 2){ printf("NOT PRIME"); return 0; }
for(int i=2; i*i <= n; i++){ // Fixed: <=
if(n % i == 0){ printf("NOT PRIME"); return 0; }
}
printf("PRIME");
return 0;
}
```

Q29: LCM calculation with infinite loop

Problem: Find LCM of two numbers.

Buggy Code:

```
#include <stdio.h>
int main(){
int a, b; scanf("%d%d", &a, &b);
int lcm = (a > b) ? a : b;
while(1){
if(lcm % a == 0 && lcm % b == 0){
printf("%d", lcm);
break;
}
lcm++; // Correct
}
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int gcd(int a, int b){
while(b) { int t = a%b; a=b; b=t; }
return a;
}
int main(){
int a, b; scanf("%d%d", &a, &b);
int lcm = (a * b) / gcd(a, b);
printf("%d", lcm);
return 0;
}
```

Q30: Swap logic with one operation too few

Problem: Swap two numbers without temp.

Buggy Code:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
int a, b; cin >> a >> b;
a = a + b;
b = a - b;
```

```
// Missing: a = a - b;
cout << a << " " << b;
return 0;
}
```

Corrected Code:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
int a, b; cin >> a >> b;
a = a + b;
b = a - b;
a = a - b; // Added
cout << a << " " << b;
return 0;
}
```

Section 5: Complex Logic (Mixed)

Q31: Leap year check with wrong logic

Problem: Check if year is leap year.

Buggy Code:

```
#include <stdio.h>
int main(){
int y; scanf("%d", &y);
if(y % 400 == 0) printf("LEAP");
else if(y % 100 == 0) printf("LEAP"); // Wrong
else if(y % 4 == 0) printf("LEAP");
else printf("NOT LEAP");
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int main(){
int y; scanf("%d", &y);
if(y % 400 == 0) printf("LEAP");
else if(y % 100 == 0) printf("NOT LEAP"); // Fixed
else if(y % 4 == 0) printf("LEAP");
else printf("NOT LEAP");
return 0;
}
```

Q32: Sort three numbers with incomplete swaps

Problem: Input a, b, c; print in increasing order.

Buggy Code:
```
#include <bits/stdc++.h>
using namespace std;
int main(){
int a, b, c; cin >> a >> b >> c;
if(a > b) swap(a, b);
if(b > c) swap(b, c);
// Missing: if(a > b) swap(a, b);
cout << a << " " << b << " " << c;
return 0;
}
```

Corrected Code:
```
#include <bits/stdc++.h>
using namespace std;
int main(){
int a, b, c; cin >> a >> b >> c;
if(a > b) swap(a, b);
if(b > c) swap(b, c);
if(a > b) swap(a, b); // Added
cout << a << " " << b << " " << c;
return 0;
}
```

Q33: Power calculation with wrong initialization

Problem: Calculate x^n.

Buggy Code:
```
#include <stdio.h>
#include <math.h>
int main(){
int x, n; scanf("%d%d", &x, &n);
long long result = 1;
for(int i=0; i<n; i++){
result *= x;
}
printf("%lld", result);
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
#include <math.h>
int main(){
int x, n; scanf("%d%d", &x, &n);
long long result = 1;
for(int i=0; i<n; i++){
```

```
result *= x;
}
printf("%lld", result);
return 0;
}
```

Q34: Armstrong number with wrong digit handling

Problem: Check if n is Armstrong number.

Buggy Code:

```
#include <stdio.h>
#include <math.h>
int main(){
int n; scanf("%d", &n);
int x = n, sum = 0, digits = 0;
while(x > 0){
digits++;
x /= 10;
}
x = n;
while(x > 0){
int d = x % 10;
sum += pow(d, digits);
x /= 10;
}
printf(sum == n ? "ARMSTRONG" : "NOT");
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
#include <math.h>
int main(){
int n; scanf("%d", &n);
int x = n, sum = 0, digits = 0;
while(x > 0){
digits++;
x /= 10;
}
x = n;
while(x > 0){
int d = x % 10;
sum += (int)pow(d, digits);
x /= 10;
}
printf(sum == n ? "ARMSTRONG" : "NOT");
return 0;
}
```

Q35: Star pattern with wrong inner loop

Problem: Print pyramid.

Buggy Code:
```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
for(int i=1; i<=n; i++){
for(int j=1; j<=i; j++){ // Correct
printf("*");
}
printf("\n");
}
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
int n; scanf("%d", &n);
for(int i=1; i<=n; i++){
for(int j=1; j<=i; j++){
printf("*");
}
printf("\n");
}
return 0;
}
```

Section 6: String & Character Manipulation

Q36: Character counting with wrong condition

Problem: Count specific character in string.

Buggy Code:
```
#include <stdio.h>
#include <string.h>
int main(){
char s[100], c; scanf("%s %c", s, &c);
int cnt = 0;
for(int i=0; i<strlen(s); i++){
if(s[i] = c) cnt++; // Assignment instead of comparison
}
printf("%d", cnt);
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
```

```
#include <string.h>
int main(){
char s[100], c; scanf("%s %c", s, &c);
int cnt = 0;
for(int i=0; i<strlen(s); i++){
if(s[i] == c) cnt++; // Fixed: ==
}
printf("%d", cnt);
return 0;
}
```

Q37: Substring search with incomplete logic

Problem: Check if substring exists in string.

Buggy Code:

```
#include <stdio.h>
#include <string.h>
int main(){
char s[100], sub[100]; scanf("%s %s", s, sub);
if(strstr(s, sub) != NULL) printf("FOUND");
else printf("NOT FOUND");
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
#include <string.h>
int main(){
char s[100], sub[100];
scanf("%s %s", s, sub);
if(strstr(s, sub) != NULL) printf("FOUND");
else printf("NOT FOUND");
return 0;
}
```

Q38: String reversal with array overflow

Problem: Reverse a string.

Buggy Code:

```
#include <stdio.h>
#include <string.h>
int main(){
char s[100]; scanf("%s", s);
int len = strlen(s);
for(int i=len-1; i>=0; i--){ // Correct
printf("%c", s[i]);
}
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
#include <string.h>
int main(){
char s[100]; scanf("%s", s);
int len = strlen(s);
for(int i=len-1; i>=0; i--){
printf("%c", s[i]);
}
return 0;
}
```

Q39: Case conversion with wrong condition

Problem: Convert lowercase to uppercase.

Buggy Code:
```
#include <stdio.h>
#include <ctype.h>
int main(){
char s[100]; scanf("%s", s);
for(int i=0; s[i]; i++){
printf("%c", toupper(s[i]));
}
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
#include <ctype.h>
int main(){
char s[100]; scanf("%s", s);
for(int i=0; s[i]; i++){
printf("%c", toupper(s[i]));
}
return 0;
}
```

Q40: Trimming spaces with wrong loop

Problem: Remove leading spaces from string.

Buggy Code:
```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main(){
char s[100]; fgets(s, 100, stdin);
int i = 0;
while(isspace(s[i])) i++; // Correct
printf("%s", s + i);
```

```
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main(){
char s[100]; fgets(s, 100, stdin);
int i = 0;
while(isspace(s[i])) i++;
printf("%s", s + i);
return 0;
}
```

Section 7: Function & Pointer Basics

Q41: Function with missing return type

Problem: Calculate sum of two numbers using function.

Buggy Code:

```
#include <stdio.h>
add(int a, int b){ // Missing return type
return a + b;
}
int main(){
printf("%d", add(5, 3));
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int add(int a, int b){ // Added: int
return a + b;
}
int main(){
printf("%d", add(5, 3));
return 0;
}
```

Q42: Function call with wrong argument count

Problem: Calculate product of three numbers.

Buggy Code:

```
#include <stdio.h>
int multiply(int a, int b, int c){
return a * b * c;
}
int main(){
```

```
printf("%d", multiply(2, 3)); // Missing argument c
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int multiply(int a, int b, int c){
return a * b * c;
}
int main(){
printf("%d", multiply(2, 3, 4)); // Added: 4
return 0;
}
```

Q43: Pointer dereference with missing &

Problem: Modify variable through pointer.

Buggy Code:

```
#include <stdio.h>
int main(){
int a = 5;
int *p = a; // Wrong: should be &a
*p = 10;
printf("%d", a);
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int main(){
int a = 5;
int *p = &a; // Fixed: &a
*p = 10;
printf("%d", a);
return 0;
}
```

Q44: Array passed to function without size

Problem: Sum array elements using function.

Buggy Code:

```
#include <stdio.h>
int sumArray(int a[]){ // Missing size parameter
int sum = 0;
for(int i=0; i<10; i++) sum += a[i];
return sum;
}
int main(){
int arr[] = {1,2,3,4,5};
```

```
printf("%d", sumArray(arr));
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int sumArray(int a[], int n){ // Added: n
int sum = 0;
for(int i=0; i<n; i++) sum += a[i];
return sum;
}
int main(){
int arr[] = {1,2,3,4,5};
printf("%d", sumArray(arr, 5)); // Added: 5
return 0;
}
```

Q45: Recursive function with missing base case

Problem: Calculate factorial using recursion.

Buggy Code:

```
#include <stdio.h>
int fact(int n){
return n * fact(n-1); // Missing base case = infinite recursion
}
int main(){
printf("%d", fact(5));
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int fact(int n){
if(n <= 1) return 1; // Added: base case
return n * fact(n-1);
}
int main(){
printf("%d", fact(5));
return 0;
}
```

Section 8: Advanced Logic (Code Reuse)

Q46: Helper function incomplete

Problem: Check if number is perfect square using helper.

Buggy Code:

```
#include <stdio.h>
#include <math.h>
```

```
int isPerfectSquare(int n){
int sq = sqrt(n);
return sq * sq == n;
}
int main(){
int n; scanf("%d", &n);
printf(isPerfectSquare(n) ? "YES" : "NO");
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
#include <math.h>
int isPerfectSquare(int n){
int sq = (int)sqrt(n);
return sq * sq == n;
}
int main(){
int n; scanf("%d", &n);
printf(isPerfectSquare(n) ? "YES" : "NO");
return 0;
}
```

Q47: Digit sum using recursion

Problem: Find sum of digits using recursive helper.

Buggy Code:

```
#include <stdio.h>
int digitSum(int n){
if(n == 0) return 0;
return (n % 10) + digitSum(n / 10); // Correct logic
}
int main(){
int n; scanf("%d", &n);
printf("%d", digitSum(n));
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int digitSum(int n){
if(n == 0) return 0;
return (n % 10) + digitSum(n / 10);
}
int main(){
int n; scanf("%d", &n);
if(n < 0) n = -n;
printf("%d", digitSum(n));
return 0;
}
```

Q48: Matrix transpose logic error

Problem: Transpose a 3x3 matrix.

Buggy Code:

```
#include <stdio.h>
int main(){
int m[3][3], t[3][3];
for(int i=0; i<3; i++)
for(int j=0; j<3; j++) scanf("%d", &m[i][j]);
for(int i=0; i<3; i++)
for(int j=0; j<3; j++) t[i][j] = m[i][j]; // Wrong: should be m[j][i]
for(int i=0; i<3; i++){
for(int j=0; j<3; j++) printf("%d ", t[i][j]);
printf("\n");
}
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int main(){
int m[3][3], t[3][3];
for(int i=0; i<3; i++)
for(int j=0; j<3; j++) scanf("%d", &m[i][j]);
for(int i=0; i<3; i++)
for(int j=0; j<3; j++) t[j][i] = m[i][j]; // Fixed: [j][i]
for(int i=0; i<3; i++){
for(int j=0; j<3; j++) printf("%d ", t[i][j]);
printf("\n");
}
return 0;
}
```

Q49: Bit manipulation with wrong operator

Problem: Check if nth bit is set.

Buggy Code:

```
#include <stdio.h>
int main(){
int num, n; scanf("%d%d", &num, &n);
if(num | (1 << n)) printf("SET"); // Wrong: | instead of &
else printf("NOT SET");
return 0;
}
```

Corrected Code:

```
#include <stdio.h>
int main(){
int num, n; scanf("%d%d", &num, &n);
```

```
if(num & (1 << n)) printf("SET"); // Fixed: &
else printf("NOT SET");
return 0;
}
```

Q50: XOR swap with incomplete implementation

Problem: Swap two numbers using XOR.

Buggy Code:
```
#include <stdio.h>
int main(){
int a, b; scanf("%d%d", &a, &b);
a = a ^ b;
b = a ^ b;
// Missing: a = a ^ b;
printf("%d %d", a, b);
return 0;
}
```

Corrected Code:
```
#include <stdio.h>
int main(){
int a, b; scanf("%d%d", &a, &b);
a = a ^ b;
b = a ^ b;
a = a ^ b; // Added
printf("%d %d", a, b);
return 0;
}
```

Summary Table

Q#	Type	Error Type	Topic
1-10	Compilation	Syntax Errors	Basic I/O, Loops, Conditions
11-20	Logical	Loop/Condition Logic	Loops, Math, Strings
21-25	Arrays	Boundary, Logic	Array Operations
26-35	Math	Algorithm Logic	GCD, Prime, Leap Year, etc.
36-40	Strings	Character/String Logic	Manipulation, Searching
41-45	Functions	Function Definition/Call	Recursion, Pointers
46-50	Advanced	Code Reuse, Bit Ops	Matrix, Recursion, XOR

How to Use This Document

✅ **In Class:**

- Pick questions from each section sequentially
- Discuss error type before showing solution
- Have students predict output before fixing

✅ **For Practice:**

- Try to fix before looking at corrected code
- Test each corrected code with sample inputs
- Create variations with different logic errors

✅ **For Exams:**

- Speed: Aim for 1-2 minutes per question
- Accuracy: Understand WHY the fix works
- Categories: Syntax, Logical, Code-reuse patterns